

Hyperledger Sawtooth organisations run a node that interacts with the Hyperledger Sawtooth network. Each node runs at least three things:

- The main validator process
- The REST service listening for requests
- One or more transaction processors

Each organisation that enters the Hyperledger Sawtooth network runs at least one node and receives transactions submitted by fellow nodes.

A transaction processor is the server-side business logic of transaction families, which acts upon network assets. Hyperledger Sawtooth supports pluggable transaction processors that are customisable based on the specific application. Each node within the Hyperledger Sawtooth network runs a transaction processor, which processes incoming transactions submitted by authorised clients.

Transaction batches are clusters of transactions that are either all committed to state or are all not committed to state. As a result, transaction batches are often described as an atomic unit of change, since a group of transactions is treated as one and is committed to the state, as one. Every single transaction in Hyperledger Sawtooth is submitted within a batch. Batches can contain as little as a single transaction.

In Hyperledger Sawtooth, the journal maintains and extends the blockchain for the validator. It is responsible for validating candidate blocks, evaluating valid blocks to determine if they are the correct chain head and for generating new blocks to extend the chain. Transaction batches arrive at the journal, where they are evaluated, validated and added to the blockchain. Additionally, the journal resolves forks, which occur due to disagreements over who commits a block. Once blocks are completed, they are delivered to the ChainController for validation and fork resolution.

The network layer is responsible for communicating between validators in a Hyperledger Sawtooth network, including performing initial connectivity, peer discovery and message handling.

The global state contains the current state of the ledger and a chain of transaction invocations. The state for all transaction families is represented on each validator.

The process of block validation on each validator ensures that the same transactions result in the same state transitions and that the resulting data is the same for all participants in the network. The state is split into namespaces, which allow flexibility for transaction family authors to define, share and reuse global state data between transaction processors.

The consensus interface in Sawtooth

Consensus in Hyperledger Sawtooth is modular, which means that the consensus algorithm can be easily modified. Hyperledger Sawtooth provides an abstract interface that supports both PBFT and Nakamoto-style algorithms. To implement a new consensus algorithm in Hyperledger Sawtooth,

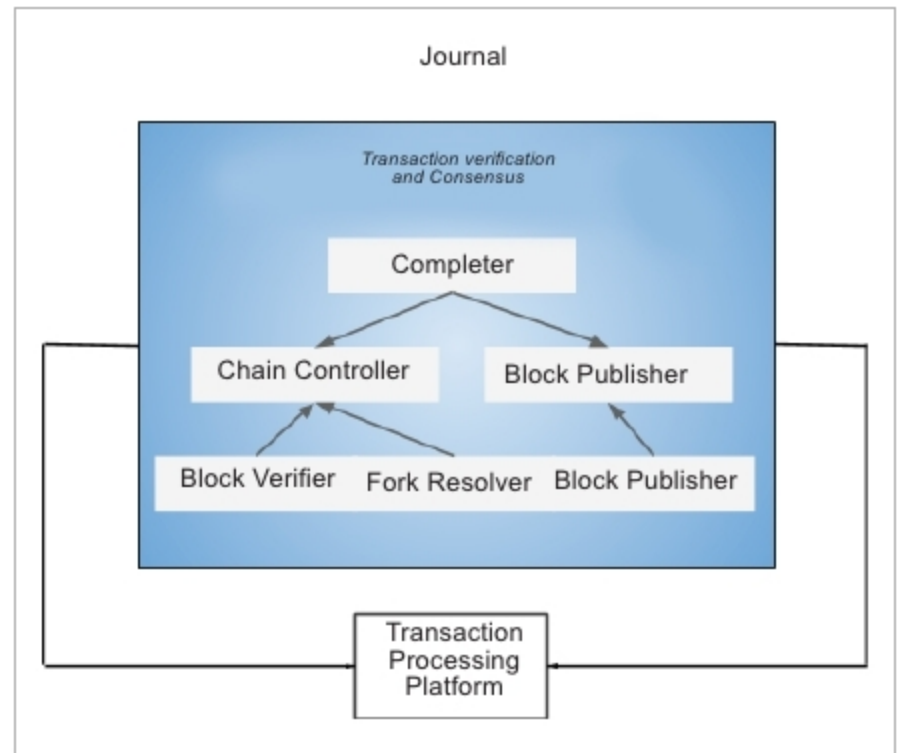


Figure 1: The consensus interface in Sawtooth

you must implement the distinct interface for the following.

- **Block publisher:** This creates new candidate blocks to extend the chain.
- **Block verifier:** This verifies that candidate blocks are published in accordance with consensus rules.
- **Fork resolver:** This chooses which fork to use as the chain head for consensus algorithms that result in a fork.

These interfaces are used by the journal component. The journal verifies that all the dependencies for the transaction batches are satisfied. When verified, completed batches are checked for validity and fork resolution and then, they are published within a block.

Introducing Proof of Elapsed Time (PoET)

The consensus algorithm commonly used in a Hyperledger Sawtooth network is the Proof of Elapsed Time, or PoET, which impartially determines who will commit a transaction to state using a lottery function that elects a leader from many different distributed nodes.

Hyperledger Sawtooth's PoET algorithm differs from the Proof of Work (PoW) algorithm implemented by the Bitcoin blockchain in that PoW relies on vast amounts of power, while PoET is able to ensure trust and randomness in electing a leader, without the high power consumption. PoET allows for increased scalability and participation, as every node in the network has an equal opportunity to create the next block on the chain.

How Proof of Elapsed Time (PoET) works

To start with, each validator within the network requests a wait time from an enclave or a trusted function. This is where the 'elapsed time' comes into play. The validator with the shortest waiting period for a specific block is appointed the leader and creates the block to be committed to the ledger. As a result, a truly random leader is elected and the